The same program was written for Python and the results are:

```
FOR_SEARCH:
       Elapsed CPU time: 16.420260511
seconds
           matches: 550
       Elapsed CPU time: 16.140975079
seconds
           matches: 550
       Elapsed CPU time: 16.49639576
seconds
           matches: 550

IN:
       Elapsed CPU time: 6.446583343
seconds
           matches: 550
       Elapsed CPU time: 6.216615487
seconds
           matches: 550
       Elapsed CPU time: 6.296716556
seconds
           matches: 550
```

From the above results, it is evident that there are no time differences between using a loop and an operator in Julia. However, the loop takes almost three times more execution time than the IN operator in Python. The interesting point here is that, in both cases, Julia has a much faster execution time than Python.

Linear regression

The next experiments were performed in machine learning algorithms. We first worked on one of the most common and simple machine learning algorithms, i.e., linear regression with a simple data set. We used a 'Head Brain' data set that contains 237 data entries and has two columns [HeadSize, BrainWeight]. In this, we had to calculate the brain weight by using the head size. So we implemented linear regression from scratch, without using any library on this data set in both Python and Julia.

Julia:

```
GC.gc()
@CPUtime begin
    linear_reg()
end
elapsed CPU time: 0.000718 seconds
```

Python:

```
gc.collect()
start = process_time()
linear_reg()
end = process_time()

print(end-start)
elapsed time: 0.007180344000000005
```

The time taken by both Julia and Python is given above.

Logistic regression

Next, we carried out an experiment on the most common type of machine learning algorithm, i.e., logistic regression, by using libraries in both languages. For Python, we used its most commonly used library sklearn while in Julia we used the GLM library. The data set that we used for this is the information about a bank's clients, which contains 10,000 data entries. The target variable is a binary variable that indicates whether the consumer left the bank (closed his or her account) or remained a customer.

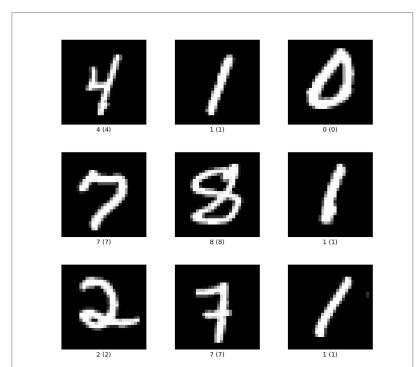

Figure 1: Example of MNIST data set

The time taken by Julia for logistic regression is given below.

```
@time log_rec()
0.027746 seconds (3.32 k allocations:
10.947 MiB)
```

The time taken by Python for logistic regression is also given below.

```
gc.collect()
start = process_time()
LogReg()
end = process_time()
print(end-start)

Accuracy : 0.8068
0.34901400000000005
```

Neural networks

After testing both languages on various programs and data sets, we tested them on neural networks and used the MNIST data set. This data set contains gray-scale images of hand-drawn digits, from zero through nine. Each image is 28x28 pixels. Each pixel value indicates the lightness or darkness of that pixel, and this value is an integer between 0 and 255, both inclusive. The data also contains a label column which represents the digit that was drawn in the respected image.

Figure 1 shows a few examples of the MNIST data set.

We created a simple neural network to test the time taken by both languages. The structure of our neural network is like this:

```
Input ---> Hidden layer ---> Output
```

```
@time Flux.train!(loss, params(model), Iterators.repeated(trainbatch,40), opt; cb = Flux.throttle(callback,10))
loss(trainbatch...) = 2.0350972755858594
  5.766363 seconds (282.95 k allocations: 1.339 GiB, 2.17% gc time, 6.92% compilation time)
```

Figure 2: Julia takes 5.76 seconds in a neural network